

Module 26: Smart Pointers & Ownership

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Part 2 Module 10 promised that smart pointers would replace raw new and delete once students understood what happens underneath; Modules 23-25 provided that understanding. This module makes ownership visible in the type system. Students use `std::unique_ptr` and `make_unique` for sole ownership, learn that it is a move-only RAI type, and distinguish transferring ownership from borrowing through references and raw pointers. Module 12's polymorphic `vector<Component*>` is rewritten with `unique_ptr`, ending the delete loop for good. `shared_ptr` and its reference count, `weak_ptr` and reference cycles, and a decision table complete the module.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain ownership, and why raw owning pointers make it invisible.
- Create and use `std::unique_ptr` with `std::make_unique`, including moving, reset and get.
- Write function signatures that transfer ownership versus borrow an object.
- Store polymorphic objects in `std::vector<std::unique_ptr<Base>>`.
- Use `std::shared_ptr` with `std::make_shared`, and explain its reference count.
- Break `shared_ptr` reference cycles with `std::weak_ptr` and `lock()`.
- Choose the right pointer (or no pointer) for a given ownership situation.

Prerequisites

- Module 25 (move semantics and move-only types).
- Part 2 Module 12 (polymorphism and virtual destructors).

Materials & Instructor Prep

- 26-smart-pointers/examples/ 01 to 05; 03_polymorphic_unique_ptr.cpp side by side with Part 2's 12-polymorphism/examples/03_polymorphic_collection.cpp.
- The decision table from the README on a slide or poster - students will refer to it for the rest of the course.

Key Vocabulary

Term	Definition
Ownership	Responsibility for deleting a heap object. Every heap object should have a clear owner.

Term	Definition
<code>std::unique_ptr</code>	A move-only smart pointer that is the sole owner of an object and deletes it automatically.
<code>std::make_unique</code> / <code>std::make_shared</code>	Functions that create an object and wrap it in a smart pointer in one step, so you never write new.
Borrowing	Using an object through a reference or raw pointer without owning it; the owner must outlive the borrow.
<code>std::shared_ptr</code>	A smart pointer that shares ownership with other <code>shared_ptr</code> s through a reference count; the object is deleted when the count reaches zero.
Reference cycle	Objects that keep each other alive through <code>shared_ptr</code> s, so their counts never reach zero and they leak.
<code>std::weak_ptr</code>	A non-owning observer of a <code>shared_ptr</code> -managed object; <code>lock()</code> gives a <code>shared_ptr</code> if the object still exists.

Lesson Outline & Instructor Talking Points

1. The ownership question (10 min)

- Show the signature `Sensor* createSensor(std::string name);` and ask the class: must the caller delete the result? Collect answers - the point is that nobody can know from the signature.
- Define ownership as responsibility for deleting, and frame smart pointers as making ownership part of the type.

2. `unique_ptr` (20 min)

- Run examples/01_unique_ptr_basics.cpp and read the `[+]/[-]` trace. Uncomment the copy line to show the compile error, then show moving.
- Connect explicitly to Module 25: `unique_ptr` is a move-only RAII class, exactly like `SerialPort`.

3. Transfer vs borrow (15 min)

- Walk through examples/02_transfer_and_borrow.cpp. For each function, have students say aloud what happens to ownership just by reading its signature.
- State the modern rule: a raw pointer or reference never owns - it borrows.

Instructor note: *This step changes how students read code. Practise it: give three more signatures and ask 'owns, borrows, or transfers?'*

4. Polymorphic containers (15 min)

- Show Part 2's Module 12 circuit and examples/03_polymorphic_unique_ptr.cpp side by side. The delete loop is gone; virtual dispatch is unchanged; the virtual destructor rule still applies.
- Introduce `auto` in range-for loops as a convenience for long type names.

5. shared_ptr, weak_ptr and choosing (20 min)

- Run examples/04_shared_ptr.cpp and follow the reference count as owners come and go.
- Run examples/05_weak_ptr_cycle.cpp and point to the missing destructor lines in the leaky version - a leak without a single new.
- Finish with the decision table, stressing 'default to unique_ptr' and 'the best pointer is often no pointer'.

Instructor note: Students often overuse shared_ptr because it 'just works'. Push back firmly: shared ownership hides who is responsible and invites cycles.

Common Student Misconceptions

Misconception	How to Address It
"Smart pointers mean I never need raw pointers."	Raw pointers and references are still right for borrowing. What disappears is raw OWNING pointers - new and delete.
"Use shared_ptr everywhere to be safe."	It hides ownership, costs more, and allows cycles that leak. Default to unique_ptr and use shared_ptr only when there is genuinely no single owner.
"Pass const std::unique_ptr<T>& to functions that use the object."	That ties the function to one ownership strategy for no benefit. Pass T& or const T& (or T* if the object may be absent).
"shared_ptr can never leak."	Two objects holding shared_ptrs to each other form a cycle whose counts never reach zero. One direction must be a weak_ptr.

Exercises & Module Project

Exercise: 26-smart-pointers/exercises/exercise1.cpp - convert a Shelter that owns raw Animal pointers to std::vector<std::unique_ptr<Animal>>, with admit() taking ownership, find() borrowing, and rehome() transferring ownership out; delete the destructor and explain the Rule of Zero. Project (three tracks): Generic - school staff (unique_ptr), students on the roll (shared_ptr) and club registers (weak_ptr); EE - appliances in rooms (unique_ptr), rooms in a house (shared_ptr) and a smart plug observing a room (weak_ptr); Biomedical - devices in wards (unique_ptr), admitted patients (shared_ptr) and devices observing patients (weak_ptr).

Facilitation guidance: The grep check in the project README (no new or delete outside comments) is the quickest correctness test. Then read the ownership map at the top of main.cpp - it shows whether a student understands owns/shares/observes/borrows, which matters more than the code compiling. Build with -fsanitize=address to confirm there are no leaks.

Discussion & Comprehension Check Questions

1. Why can't a unique_ptr be copied?

Guidance: Two unique_ptrs owning the same object would both delete it. Copying is deleted; ownership can only be moved.

2. What does a function taking std::unique_ptr<Sensor> by value tell its caller?

Guidance: That it takes ownership: the caller must `std::move` the pointer in and will no longer own (or be able to use) the sensor.

3. How does `weak_ptr` break a reference cycle?

Guidance: It refers to the object without increasing its reference count, so the owning `shared_ptrs` can reach zero; `lock()` reports whether the object still exists.

Assessment Ideas

- Given five function signatures, students classify each parameter or return type as owning, sharing, observing or borrowing.
- Students rewrite a short raw-pointer program with smart pointers and no `new/delete`, then explain each pointer choice.

Answer Key Notes

See `26-smart-pointers/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "We can now say precisely who owns what. Next module asks a design question: when one object 'has' another, is it composition, aggregation or association - and how does each look in C++?"

Instructor Tips

- If time is short, protect transfer-vs-borrow (step 3) and polymorphic containers (step 4); `weak_ptr` can be shortened to the cycle demonstration.
- From this module on, gently refuse to accept `new/delete` in any student project code.